

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 5
IMPLEMENTASI STRUKTUR DATA SINGLE LINKED LIST



Disusun Oleh:
Annisa Layli Ramadhani
2511532024

Dosen Pengampu: Wahyudi. Dr.. S.T.M.T
Asisten Praktikum: Muhammad Galid Avero

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia nya laporan praktikum struktur data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas pelaksanaan praktikum yang telah dilakukan, khususnya pada materi Single Linked List.

Laporan ini bertujuan untuk memahami konsep dasar Single Linked List, termasuk struktur node, penggunaan pointer, serta implementasi operasi dasar seperti penambahan, penghapusan, dan penelusuran data. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang.

Padang, 5 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang	4
1.2 Tujuan	4
1.3 Manfaat	5
BAB II PEMBAHASAN	6
2.1 Dasar Teori	6
2.2 Langkah Kerja.....	7
2.2.1 Kode NodeSLL_2511532024	7
2.2.2 Penjelasan Kode NodeSLL_2511532024.....	7
2.2.3 Kode TambahSLL_2511532024.....	8
2.2.4 Penjelasan Kode TambahSLL_2511532024	10
2.2.5 Kode PencarianSLL_2511532024.....	14
2.2.6 Penjelasan Kode PencarianSLL_2511532024	15
2.2.7 Kode HapusSLL_2511532024	18
2.2.8 Penjelasan Kode HapusSLL_2511532024	20
2.2.9 Output Kode Program.....	25
BAB III PENUTUP	26
3.1 Kesimpulan	26
DAFTAR PUSTAKA	27
TUGAS PRAKTIKUM STRUKTUR DATA	28
4.1 Soal Tugas	28
4.2 Kode program.....	28
4.2.1 Class Pasien_2511532024	28
4.2.2 Class RumahSakit_2511532024	29
4.3 Penjelasan.....	32
4.3.1 Class Pasien_2511532024	32
4.3.2 Class RumahSakit_2511532024	34
4.4 Output Kode Program	40

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya pengolahan data yang efisien dan terstruktur. Dalam dunia pemrograman, struktur data menjadi salah satu komponen penting yang berperan dalam mengatur, menyimpan, serta memanipulasi data agar dapat digunakan secara optimal. Pemilihan struktur data yang tepat akan sangat mempengaruhi kinerja suatu program, baik dari segi waktu eksekusi maupun penggunaan memori.

Salah satu struktur data yang sering digunakan adalah linked list, khususnya single linked list. Struktur data ini terdiri dari sekumpulan node yang saling terhubung melalui pointer, dimana setiap node menyimpan data serta referensi ke node berikutnya. Berbeda dengan array yang memiliki ukuran tetap dan penyimpanan berurutan di memori, single linked list bersifat dinamis sehingga memungkinkan penambahan dan penghapusan data dilakukan dengan lebih fleksibel.

Pemahaman terhadap konsep single linked list sangat penting bagi mahasiswa, terutama dalam mempelajari bagaimana data dapat dikelola secara dinamis menggunakan referensi atau pointer. Selain itu, konsep ini juga menjadi dasar bagi struktur data lain yang lebih kompleks, seperti double linked list, stack, queue, dan tree.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Memahami konsep dasar struktur data single linked list.
2. Mengetahui cara kerja node dan pointer dalam menghubungkan data.
3. Mampu mengimplementasikan operasi dasar pada single linked list, seperti penambahan, penghapusan, dan penelusuran data.
4. Melatih kemampuan pemrograman dalam mengelola struktur data secara dinamis.

1.3 Manfaat

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut:

1. Meningkatkan pemahaman mahasiswa terhadap konsep struktur data, khususnya single linked list.
2. Mengembangkan kemampuan dalam merancang dan mengimplementasikan program berbasis struktur data.
3. Menjadi dasar dalam mempelajari struktur data yang lebih kompleks.
4. Membantu mahasiswa dalam menyelesaikan permasalahan yang berkaitan dengan pengolahan data secara efisien.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Struktur data merupakan cara untuk menyimpan dan mengorganisasikan data sehingga dapat digunakan secara efisien. Dalam pemrograman, pemilihan struktur data yang tepat sangat berpengaruh terhadap kinerja program, baik dari segi kecepatan maupun penggunaan memori.

Salah satu struktur data yang banyak digunakan adalah linked list. Linked list adalah sekumpulan elemen data yang disebut node, dimana setiap node terdiri dari dua bagian, yaitu data dan referensi (pointer) yang menunjuk ke node berikutnya. Berbeda dengan array, linked list tidak disimpan secara berurutan di dalam memori, sehingga ukurannya dapat berubah secara dinamis sesuai kebutuhan.

Single linked list merupakan bentuk paling sederhana dari linked list, dimana setiap node hanya memiliki satu pointer yang mengarah ke node berikutnya. Node pertama disebut sebagai head, sedangkan node terakhir akan menunjuk ke nilai null yang menandakan akhir dari list. Struktur ini memungkinkan proses penambahan dan penghapusan data dilakukan dengan lebih fleksibel dibandingkan array.

Beberapa operasi dasar pada single linked list antara lain:

1. Insert (penambahan data), yaitu menambahkan node baru ke dalam list, baik di awal, tengah, maupun akhir.
2. Delete (penghapusan data), yaitu menghapus node tertentu dari list.
3. Traversal (penelusuran), yaitu mengakses setiap node secara berurutan dari awal hingga akhir.
4. Search (pencarian), yaitu mencari data tertentu dalam list.

Dalam implementasinya, single linked list memanfaatkan konsep reference atau pointer. Pointer digunakan untuk menyimpan alamat dari node berikutnya, sehingga setiap node saling terhubung membentuk

suatu rangkaian. Jika pointer bernilai null, maka memanandakan bahwa node tersebut adalah node terakhir.

2.2 Langkah Kerja

2.2.1 Kode NodeSLL_2511532024

```
1. public class NodeSLL_2511532024 {  
2.     // node bagian data  
3.     int data_2024;  
4.     // pointer ke node berikutnya  
5.     NodeSLL_2511532024 next_2024;  
6.     // konstruktor menginisialisasi node dengan data  
7.     public NodeSLL_2511532024(int data_2024) {  
8.         this.data_2024 = data_2024;  
9.         this.next_2024 = null;  
10.    }  
11. }
```

2.2.2 Penjelasan Kode NodeSLL_2511532024

1. Deklarasi atribut (data dan pointer)

```
int data_2024;  
NodeSLL_2511532024 next_2024;
```

Bagian ini mendefinisikan dua komponen utama dalam node. data_2024 digunakan untuk menyimpan nilai data bertipe integer, sedangkan next_2024 merupakan pointer atau referensi yang menunjuk ke node berikutnya. Jika bernilai null, berarti node tersebut adalah node terakhir dalam linked list.

2. Konstruktor node

```
public NodeSLL_2511532024(int data_2024) {  
    this.data_2024 = data_2024;  
    this.next_2024 = null;  
}
```

Konstruktor ini berfungsi untuk menginisialisasi node saat pertama kali dibuat. Nilai data_2024 diisi sesuai parameter yang diberikan, sedangkan next_2024 diatur null sebagai

kondisi awal, menandakan node belum terhubung dengan node lain.

2.2.3 Kode TambahSLL_2511532024

```
1. public class TambahSLL_2511532024 {
2.     public static NodeSLL_2511532024
       insertAtFront_2024(NodeSLL_2511532024 head_2024, int
       value_2024) {
3.         NodeSLL_2511532024 new_node_2024 = new
       NodeSLL_2511532024(value_2024);
4.         new_node_2024.next_2024 = head_2024;
5.         return new_node_2024;
6.     }
7.     // fungsi menambahkan node di akhir SLL
8.     public static NodeSLL_2511532024
       insertAtEnd_2024(NodeSLL_2511532024 head_2024, int
       value_2024) {
9.         // buat sebuah node dengan sebuah nilai
10.        NodeSLL_2511532024 newNode_2024 = new
       NodeSLL_2511532024(value_2024);
11.        // jika list kosong maka node jadi head
12.        if (head_2024 == null) {
13.            return newNode_2024;
14.        }
15.        // simpan head ke variabel sementara
16.        NodeSLL_2511532024 last_2024 = head_2024;
17.        // telusuri ke node akhir
18.        while (last_2024.next_2024 != null) {
19.            last_2024 = last_2024.next_2024;
20.        }
21.        // ubah pointer
22.        last_2024.next_2024 = newNode_2024;
23.        return head_2024;
24.    }
25.    static NodeSLL_2511532024 GetNode_2024(int data_2024)
       {
26.        return new NodeSLL_2511532024(data_2024);
27.    }
28.
29.    static NodeSLL_2511532024
       insertPos_2024(NodeSLL_2511532024 headNode_2024, int
       position_2024, int value_2024) {
30.        NodeSLL_2511532024 head_2024 = headNode_2024;
31.        if (position_2024 < 1)
32.            System.out.print("Invalid position");
33.        if (position_2024 == 1) {
34.            NodeSLL_2511532024 new_node_2024 = new
       NodeSLL_2511532024(value_2024);
35.            new_node_2024.next_2024 = head_2024;
36.            return new_node_2024;
37.        } else {
38.            while (position_2024-- != 0) {
39.                if (position_2024 == 1) {
```



```

40.         NodeSLL_2511532024
    newNode_2024 = GetNode_2024(value_2024);
41.         newNode_2024.next_2024 =
    headNode_2024.next_2024;
42.         headNode_2024.next_2024 =
    newNode_2024;
43.         break;
44.     }
45.     headNode_2024 =
    headNode_2024.next_2024;
46. }
47.     if (position_2024 != 1)
48.         System.out.print("Posisi di luar
    jangkauan");
49.         return head_2024;
50.     }
51. }
52.
53.     public static void printList_2024
    (NodeSLL_2511532024 head_2024) {
54.         NodeSLL_2511532024 curr_2024 =
    head_2024;
55.         while (curr_2024.next_2024 != null) {
56.             System.out.print(curr_2024.data_2024+" --> ");
57.             curr_2024 = curr_2024.next_2024;
58.         }
59.         if (curr_2024.next_2024 == null) {
60.             System.out.print(curr_2024.data_2024);
61.             System.out.println();
62.         }
63.     }
64.
65.     public static void main(String[] args) {
66.         // buat linked list 2->3->5->6
67.         NodeSLL_2511532024 head_2024 = new
    NodeSLL_2511532024(2);
68.         head_2024.next_2024 = new
    NodeSLL_2511532024(3);
69.         head_2024.next_2024.next_2024 = new
    NodeSLL_2511532024(5);
70.         head_2024.next_2024.next_2024.next_2024
    = new NodeSLL_2511532024(6);
71.         // cetak list asli
72.         System.out.print("Senarai berantai awal:
    ");
73.         printList_2024(head_2024);
74.         // tambahkan node baru di depan
75.         System.out.print("tambah 1 simpul di
    depan: ");
76.         int data_2024 = 1;
77.         head_2024 =
    insertAtFront_2024(head_2024, data_2024);
78.         // cetak update list
79.         printList_2024(head_2024);
80.         // tambahkan node baru di belakang

```

```

81.         System.out.print("tambah 1 simpul di
           belakang: ");
82.         int data2_2024 = 7;
83.         head_2024 = insertAtEnd_2024(head_2024,
           data2_2024);
84.         // cetak update list
85.         printList_2024(head_2024);
86.         System.out.print("tambah 1 simpul ke
           data 4: ");
87.         int data3_2024 = 4;
88.         int pos_2024 = 4;
89.         head_2024 = insertPos_2024(head_2024,
           pos_2024, data3_2024);
90.         // cetak update list
91.         printList_2024(head_2024);
92.     }
93.
94. }

```

2.2.4 Penjelasan Kode TambahSLL_2511532024

1. Deklarasi kelas

```
public class TambahSLL_2511532024 {
```

Bagian ini mendefinisikan kelas yang berisi kumpulan method untuk melakukan operasi pada Single Linked List. Kelas ini bersifat public sehingga bisa digunakan oleh kelas lain dalam program.

2. Method insert di depan

```

public static NodeSLL_2511532024
insertAtFront_2024(NodeSLL_2511532024 head_2024, int
value_2024) {
    NodeSLL_2511532024 new_node_2024 = new
NodeSLL_2511532024(value_2024);
    new_node_2024.next_2024 = head_2024;
    return new_node_2024;
}

```

Method ini digunakan untuk menambahkan node di bagian depan linked list. Node baru dibuat terlebih dahulu, kemudian pointer next_2024 diarahkan ke head lama.

Setelah itu, node baru dikembalikan sebagai head yang baru karena posisinya sekarang berada di awal list.

3. Method insert di belakang

```
public static NodeSLL_2511532024
insertAtEnd_2024(NodeSLL_2511532024 head_2024, int
value_2024) {
    NodeSLL_2511532024 newNode_2024 = new
NodeSLL_2511532024(value_2024);
    if (head_2024 == null) {
        return newNode_2024;
    }
    NodeSLL_2511532024 last_2024 = head_2024;
    while (last_2024.next_2024 != null) {
        last_2024 = last_2024.next_2024;
    }
    last_2024.next_2024 = newNode_2024;
    return head_2024;
}
```

Method ini berfungsi untuk menambahkan node di akhir linked list. Jika list kosong, node baru langsung menjadi head. Jika tidak, program menelusuri hingga node terakhir menggunakan perulangan, kemudian pointer node terakhir diarahkan ke node baru. Head tetap dikembalikan karena tidak berubah.

4. Method pembuat node

```
static NodeSLL_2511532024 GetNode_2024(int data_2024)
{
    return new NodeSLL_2511532024(data_2024);
}
```

Method ini berfungsi untuk membuat node baru secara sederhana. Isinya hanya mengembalikan objek node berdasarkan nilai yang diberikan, sehingga membantu menyederhanakan penulisan kode pada method lain.

5. Method insert di posisi tertentu

```
static NodeSLL_2511532024
insertPos_2024(NodeSLL_2511532024 headNode_2024, int
position_2024, int value_2024) {
    NodeSLL_2511532024 head_2024 = headNode_2024;
    if (position_2024 < 1)
        System.out.print("Invalid position");
    if (position_2024 == 1) {
        NodeSLL_2511532024 new_node_2024 = new
NodeSLL_2511532024(value_2024);
        new_node_2024.next_2024 = head_2024;
        return new_node_2024;
    } else {
        while (position_2024-- != 0) {
            if (position_2024 == 1) {
                NodeSLL_2511532024 newNode_2024 =
GetNode_2024(value_2024);
                newNode_2024.next_2024 =
headNode_2024.next_2024;
                headNode_2024.next_2024 = newNode_2024;
                break;
            }
            headNode_2024 = headNode_2024.next_2024;
        }
        if (position_2024 != 1)
            System.out.print("Posisi di luar jangkauan");
        return head_2024;
    }
}
```

```
}  
}
```

Method ini digunakan untuk menambahkan node pada posisi tertentu dalam linked list. Jika posisi 1, maka node ditambahkan di depan. Jika posisi lebih besar, program menelusuri list hingga posisi yang diinginkan, lalu menyisipkan node baru dengan mengatur pointer agar tetap terhubung. Jika posisi tidak valid atau melebihi panjang list, akan ditampilkan pesan kesalahan.

6. Method menampilkan linked list

```
public static void printList_2024 (NodeSLL_2511532024  
head_2024) {  
    NodeSLL_2511532024 curr_2024 = head_2024;  
    while (curr_2024.next_2024 != null) {  
        System.out.print(curr_2024.data_2024+" --> ");  
        curr_2024 = curr_2024.next_2024;  
    }  
    if (curr_2024.next_2024 == null) {  
        System.out.print(curr_2024.data_2024);  
        System.out.println();  
    }  
}
```

Method ini digunakan untuk menampilkan seluruh isi linked list. Program menelusuri node dari head hingga akhir, mencetak setiap data dengan tanda panah. Node terakhir dicetak tanpa tanda panah agar hasil tampil lebih rapi.

7. Method utama

```
public static void main(String[] args) {  
    NodeSLL_2511532024 head_2024 = new  
    NodeSLL_2511532024(2);
```

```

        head_2024.next_2024 = new NodeSLL_2511532024(3);
        head_2024.next_2024.next_2024 = new
NodeSLL_2511532024(5);
        head_2024.next_2024.next_2024.next_2024 = new
NodeSLL_2511532024(6);

        System.out.print("Senarai berantai awal: ");
        printList_2024(head_2024);

        System.out.print("tambah 1 simpul di depan: ");
        head_2024 = insertAtFront_2024(head_2024, 1);
        printList_2024(head_2024);

        System.out.print("tambah 1 simpul di belakang: ");
        head_2024 = insertAtEnd_2024(head_2024, 7);
        printList_2024(head_2024);

        System.out.print("tambah 1 simpul ke data 4: ");
        head_2024 = insertPos_2024(head_2024, 4, 4);
        printList_2024(head_2024);
    }

```

Bagian ini merupakan program utama yang menjalankan seluruh proses. Pertama dibuat linked list awal secara manual, kemudian ditampilkan. Setelah itu dilakukan beberapa operasi penambahan, yaitu di depan, di belakang, dan pada posisi tertentu. Setiap perubahan ditampilkan untuk menunjukkan hasil dari operasi yang dilakukan.

2.2.5 Kode PencarianSLL_2511532024

```

1. public class PencarianSLL_2511532024 {
2.     static boolean searchKey_2024 (NodeSLL_2511532024
   head_2024, int key_2024) {
3.         NodeSLL_2511532024 curr_2024 = head_2024;
4.         while (curr_2024 != null) {
5.             if (curr_2024.data_2024 == key_2024)

```

```

6.         return true;
7.         curr_2024 = curr_2024.next_2024;
8.     }
9.         return false;
10. }
11.
12. public static void traversal_2024(NodeSLL_2511532024
    head_2024) {
13.     // mulai dari head
14.     NodeSLL_2511532024 curr_2024 = head_2024;
15.     // telusuri sampai pointer null
16.     while (curr_2024 != null) {
17.         System.out.print(" " +
            curr_2024.data_2024);
18.         curr_2024 = curr_2024.next_2024;
19.     }
20.     System.out.println();
21. }
22.
23. public static void main(String[] args) {
24.     NodeSLL_2511532024 head_2024 = new
        NodeSLL_2511532024(14);
25.     head_2024.next_2024 = new
        NodeSLL_2511532024(21);
26.     head_2024.next_2024.next_2024 = new
        NodeSLL_2511532024(13);
27.     head_2024.next_2024.next_2024.next_2024 = new
        NodeSLL_2511532024(30);
28.     head_2024.next_2024.next_2024.next_2024.next_2024 =
        new NodeSLL_2511532024(10);
29.     System.out.print("Penelusuran SLL : ");
30.     traversal_2024(head_2024);
31.     // data yang akan dicari
32.     int key_2024 = 30;
33.     System.out.print("cari data " + key_2024 + "
        = ");
34.     if (searchKey_2024(head_2024, key_2024))
35.         System.out.println("ketemu");
36.     else
37.         System.out.println("tidak ada");
38. }
39. }

```

2.2.6 Penjelasan Kode PencarianSLL_2511532024

1. Deklarasi kelas

```
public class PencarianSLL_2511532024 {
```

Bagian ini mendefinisikan kelas yang berisi method untuk melakukan operasi pencarian dan penelusuran pada Single Linked List. Kelas ini bersifat public sehingga dapat

digunakan sebagai program utama atau dipanggil dari kelas lain.

2. Method pencarian data

```
static boolean searchKey_2024 (NodeSLL_2511532024
head_2024, int key_2024) {
    NodeSLL_2511532024 curr_2024 = head_2024;
    while (curr_2024 != null) {
        if (curr_2024.data_2024 == key_2024)
            return true;
        curr_2024 = curr_2024.next_2024;
    }
    return false;
}
```

Method ini digunakan untuk mencari suatu nilai (key_2024) dalam linked list. Proses dimulai dari node pertama (head_2024) menggunakan variabel bantu curr_2024. Selama node tidak bernilai null, program akan membandingkan data pada setiap node dengan nilai yang dicari. Jika ditemukan kecocokan, method langsung mengembalikan nilai true. Jika seluruh node sudah ditelusuri dan tidak ditemukan, maka method mengembalikan false.

3. Method penelusurann

```
public static void traversal_2024(NodeSLL_2511532024
head_2024) {
    NodeSLL_2511532024 curr_2024 = head_2024;
    while (curr_2024 != null) {
        System.out.print(" " + curr_2024.data_2024);
        curr_2024 = curr_2024.next_2024;
    }
    System.out.println();
}
```



```
}
```

Method ini berfungsi untuk menampilkan seluruh isi linked list secara berurutan. Penelusuran dimulai dari head dan dilakukan hingga mencapai node terakhir (saat pointer bernilai null). Setiap data node dicetak ke layar, sehingga menghasilkan tampilan seluruh elemen dalam list.

4. Method utama (main)

```
public static void main(String[] args) {  
    NodeSLL_2511532024 head_2024 = new  
NodeSLL_2511532024(14);  
    head_2024.next_2024 = new NodeSLL_2511532024(21);  
    head_2024.next_2024.next_2024 = new  
NodeSLL_2511532024(13);  
    head_2024.next_2024.next_2024.next_2024 = new  
NodeSLL_2511532024(30);  
    head_2024.next_2024.next_2024.next_2024.next_2024 =  
new NodeSLL_2511532024(10);  
  
    System.out.print("Penelusuran SLL : ");  
    traversal_2024(head_2024);  
  
    int key_2024 = 30;  
    System.out.print("cari data " + key_2024 + " = ");  
    if (searchKey_2024(head_2024, key_2024))  
        System.out.println("ketemu");  
    else  
        System.out.println("tidak ada");  
}
```

Bagian ini merupakan program utama yang menjalankan seluruh proses. Pertama dibuat linked list secara manual dengan beberapa nilai. Setelah itu, method traversal_2024()

dipanggil untuk menampilkan isi list. Selanjutnya ditentukan nilai yang ingin dicari, kemudian method searchKey_2024() digunakan untuk mengecek apakah nilai tersebut ada di dalam list. Hasil pencarian ditampilkan dalam bentuk “ketemu” jika ada, atau “tidak ada” jika tidak ditemukan.

2.2.7 Kode HapusSLL_2511532024

```
1. public class HapusSLL_2511532024 {
2. // fungsi untuk menghapus head
3. public static NodeSLL_2511532024
   deleteHead_2024(NodeSLL_2511532024 head_2024) {
4.     // jika SLL kosong
5.     if (head_2024 == null)
6.         return null;
7.     //pindahkan head ke node berikutnya
8.     head_2024 = head_2024.next_2024;
9.     // return head baru
10.    return head_2024;
11. }
12.
13. // fungsi menghapus node terakhir SLL
14. public static NodeSLL_2511532024
   removeLastNode_2024(NodeSLL_2511532024 head_2024) {
15.     // jika list kosong, return null
16.     if (head_2024 == null) {
17.         return null;
18.     }
19.     // jika list satu node, hapus node dan return
   null
20.     if (head_2024.next_2024 == null) {
21.         return null;
22.     }
23.     // temukan node terakhir ke dua
24.     NodeSLL_2511532024 secondLast_2024 =
   head_2024;
25.     while (secondLast_2024.next_2024.next_2024 !=
   null) {
26.         secondLast_2024 =
   secondLast_2024.next_2024;
27.     }
28.     // hapus node terakhir
29.     secondLast_2024.next_2024 = null;
30.     return head_2024;
31. }
32.
33. // fungsi menghapus node di posisi tertentu
34. public static NodeSLL_2511532024
   deleteNode_2024(NodeSLL_2511532024 head_2024, int
   position_2024) {
35.     NodeSLL_2511532024 temp_2024 = head_2024;
36.     NodeSLL_2511532024 prev_2024 = null;
37.     // jika linked list null
```

```

38.         if (temp_2024 == null)
39.             return head_2024;
40.         // kasus 1: head dihapus
41.         if (position_2024 == 1) {
42.             head_2024 = temp_2024.next_2024;
43.             return head_2024;
44.         }
45.         // kasus 2: menghapus node di tengah
46.         // telusuri ke node yang dihapus
47.         for (int i_2024 = 1; temp_2024 != null &&
i_2024 < position_2024; i_2024++) {
48.             prev_2024 = temp_2024;
49.             temp_2024 = temp_2024.next_2024;
50.         }
51.         // jika ditemukan, hapus node
52.         if (temp_2024 != null) {
53.             prev_2024.next_2024 =
temp_2024.next_2024;
54.         } else {
55.             System.out.println("Data tidak Ada");
56.             return head_2024;
57.         }
58.         return prev_2024;
59.     }
60.     // fungsi mencetak SLL
61.     public static void printList_2024(NodeSLL_2511532024
head_2024) {
62.         NodeSLL_2511532024 curr_2024 = head_2024;
63.         while (curr_2024.next_2024 != null) {
64.             System.out.print(curr_2024.data_2024 +
" --> ");
65.             curr_2024 = curr_2024.next_2024;
66.         }
67.         if (curr_2024.next_2024 == null) {
68.             System.out.print(curr_2024.data_2024);
69.             System.out.println();
70.         }
71.     }
72.     // kelas main
73.     public static void main(String[] args) {
74.         // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 ->
null
75.         NodeSLL_2511532024 head_2024 = new
NodeSLL_2511532024(1);
76.         head_2024.next_2024 = new
NodeSLL_2511532024(2);
77.         head_2024.next_2024.next_2024 = new
NodeSLL_2511532024(3);
78.         head_2024.next_2024.next_2024.next_2024 = new
NodeSLL_2511532024(4);
79.         head_2024.next_2024.next_2024.next_2024.next_2024 =
new NodeSLL_2511532024(5);
80.         head_2024.next_2024.next_2024.next_2024.next_2024.ne
xt_2024 = new NodeSLL_2511532024(6);
81.         // cetak list awal

```

```

82.     System.out.println("List awal: ");
83.     printList_2024(head_2024);
84.     // hapus head
85.     head_2024 = deleteHead_2024(head_2024);
86.     System.out.println("List setelah head dihapus:
");
87.     printList_2024(head_2024);
88.     // hapus node terakhir
89.     head_2024 = removeLastNode_2024(head_2024);
90.     System.out.println("List setelah simpul
    terakhir di hapus ");
91.     printList_2024(head_2024);
92.     // deleting node at position 2
93.     int position_2024 = 2;
94.     head_2024 = deleteNode_2024(head_2024,
    position_2024);
95.     // print list after deletion
96.     System.out.println("List setelah posisi 2
    dihapus: ");
97.     printList_2024(head_2024);
98. }
99. }

```

2.2.8 Penjelasan Kode HapusSLL_2511532024

1. Deklarasi kelas

```
public class HapusSLL_2511532024 {
```

Bagian ini mendefinisikan kelas yang berisi berbagai method untuk operasi penghapusan pada Single Linked List. Kelas ini menjadi tempat semua fungsi delete dikumpulkan agar terorganisir.

2. Method hapus head

```

public static NodeSLL_2511532024
deleteHead_2024(NodeSLL_2511532024 head_2024) {
    if (head_2024 == null)
        return null;
    head_2024 = head_2024.next_2024;
    return head_2024;
}

```

Method ini digunakan untuk menghapus node pertama (head). Jika list kosong, langsung mengembalikan null. Jika tidak, head digeser ke node berikutnya, sehingga node lama

otomatis “terlepas” dari list. Head baru kemudian dikembalikan.

3. Method hapus node terakhir

```
public static NodeSLL_2511532024
removeLastNode_2024(NodeSLL_2511532024 head_2024)
{
    if (head_2024 == null) {
        return null;
    }
    if (head_2024.next_2024 == null) {
        return null;
    }
    NodeSLL_2511532024 secondLast_2024 = head_2024;
    while (secondLast_2024.next_2024.next_2024 != null) {
        secondLast_2024 = secondLast_2024.next_2024;
    }
    secondLast_2024.next_2024 = null;
    return head_2024;
}
```

Method ini berfungsi untuk menghapus node terakhir. Jika list kosong atau hanya memiliki satu node, maka hasilnya null. Jika lebih dari satu node, program menelusuri hingga node kedua terakhir, lalu memutus hubungan ke node terakhir dengan mengatur pointer menjadi null. Head tetap dikembalikan.

4. Method hapus di posisi tertentu

```
public static NodeSLL_2511532024
deleteNode_2024(NodeSLL_2511532024 head_2024, int
position_2024) {
    NodeSLL_2511532024 temp_2024 = head_2024;
```

```

NodeSL_2511532024 prev_2024 = null;
if (temp_2024 == null)
    return head_2024;
if (position_2024 == 1) {
    head_2024 = temp_2024.next_2024;
    return head_2024;
}
for (int i_2024 = 1; temp_2024 != null && i_2024 <
position_2024; i_2024++) {
    prev_2024 = temp_2024;
    temp_2024 = temp_2024.next_2024;
}
if (temp_2024 != null) {
    prev_2024.next_2024 = temp_2024.next_2024;
} else {
    System.out.println("Data tidak Ada");
    return head_2024;
}
return prev_2024;
}

```

Method ini digunakan untuk menghapus node berdasarkan posisi tertentu. Proses dimulai dengan dua pointer, yaitu temp_2024 untuk node yang sedang dicek dan prev_2024 untuk node sebelumnya. Jika list kosong, maka tidak ada yang dihapus. Jika posisi adalah 1, maka sama seperti menghapus head, yaitu menggeser head ke node berikutnya. Untuk posisi selain itu, program menelusuri list menggunakan perulangan hingga mencapai posisi yang dimaksud. Jika node ditemukan, maka pointer dari node sebelumnya diarahkan ke node setelah node yang dihapus, sehingga node tersebut terlepas dari list. Jika posisi melebihi panjang list, akan ditampilkan pesan bahwa data tidak ada.

Perlu diperhatikan, method ini mengembalikan prev_2024, yang sebenarnya agak janggal karena biasanya yang dikembalikan adalah head_2024.

5. Method menampilkan list

```
public static void printList_2024(NodeSLL_2511532024
head_2024) {
    NodeSLL_2511532024 curr_2024 = head_2024;

    while (curr_2024.next_2024 != null) {
        System.out.print(curr_2024.data_2024 + " --> ");
        curr_2024 = curr_2024.next_2024;
    }

    if (curr_2024.next_2024 == null) {
        System.out.print(curr_2024.data_2024);
        System.out.println();
    }
}
```

Method ini digunakan untuk mencetak seluruh isi linked list. Program menelusuri node dari head hingga node terakhir, menampilkan setiap data dengan tanda panah. Node terakhir dicetak tanpa panah agar tampilan lebih rapi.

6. Method utama

```
public static void main(String[] args) {
    NodeSLL_2511532024 head_2024 = new
NodeSLL_2511532024(1);
    head_2024.next_2024 = new NodeSLL_2511532024(2);
    head_2024.next_2024.next_2024 = new
NodeSLL_2511532024(3);
    head_2024.next_2024.next_2024.next_2024 = new
```

```

NodeSLL_2511532024(4);
    head_2024.next_2024.next_2024.next_2024.next_2024 =
new NodeSLL_2511532024(5);

head_2024.next_2024.next_2024.next_2024.next_2024.next
_2024 = new NodeSLL_2511532024(6);

System.out.println("List awal: ");
printList_2024(head_2024);

head_2024 = deleteHead_2024(head_2024);
System.out.println("List setelah head dihapus: ");
printList_2024(head_2024);

head_2024 = removeLastNode_2024(head_2024);
System.out.println("List setelah simpul terakhir di hapus
");
printList_2024(head_2024);

int position_2024 = 2;
head_2024 = deleteNode_2024(head_2024,
position_2024);

System.out.println("List setelah posisi 2 dihapus: ");
printList_2024(head_2024);
}

```

Pada bagian ini dibuat linked list secara manual dengan nilai 1 sampai 6. Setelah itu dilakukan beberapa operasi penghapusan, yaitu menghapus head, menghapus node terakhir, dan menghapus node pada posisi tertentu. Setiap perubahan langsung ditampilkan menggunakan method printList_2024 agar terlihat hasilnya.

2.2.9 Output Kode Program

1. Output Kode Program TambahSLL_2511532024

```
Senarai berantai awal: 2 --> 3 --> 5 --> 6  
tambah 1 simpul di depan: 1 --> 2 --> 3 --> 5 --> 6  
tambah 1 simpul di belakang: 1 --> 2 --> 3 --> 5 --> 6 --> 7  
tambah 1 simpul ke data 4: 1 --> 2 --> 3 --> 4 --> 5 --> 6 --> 7
```

2. Output Kode Program PencarianSLL_2511532024

```
Penelusuran SLL : 14 21 13 30 10  
cari data 30 = ketemu
```

3. Output Kode Program HapusSLL_2511532024

```
List awal:  
1 --> 2 --> 3 --> 4 --> 5 --> 6  
List setelah head dihapus:  
2 --> 3 --> 4 --> 5 --> 6  
List setelah simpul terakhir di hapus  
2 --> 3 --> 4 --> 5  
List setelah posisi 2 dihapus:  
2 --> 4 --> 5
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa single linked list merupakan salah satu struktur data dinamis yang memungkinkan pengelolaan data secara fleksibel melalui penggunaan node berikutnya, sehingga membentuk suatu rangkaian yang saling terhubung.

Melalui implementasi yang telah dilakukan, dapat dipahami bahwa operasi dasar pada single linked list meliputi penambahan data (di depan, di belakang, dan posisi tertentu), penghapusan data (head, node terakhir, dan posisi tertentu), pencarian data, serta penelusuran seluruh elemen dalam list. Semua operasi tersebut dilakukan dengan memanipulasi pointer, yang menjadi inti utama dari struktur data ini.

Selain itu, praktikum ini juga menunjukkan bahwa penggunaan single linked list lebih fleksibel dibandingkan array, terutama dalam hal penambahan dan penghapusan data, karena tidak memerlukan pergeseran elemen. Namun, akses data tidak dapat dilakukan secara langsung dan harus melalui proses penelusuran dari node awal.

DAFTAR PUSTAKA

- [1] Wahyudi, Dr., S.T., M.T., “Struktur Data: Single Linked List,” Program Studi Informatika, Universitas Andalas, 2026.

TUGAS PRAKTIKUM STRUKTUR DATA

4.1 Soal Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

4.2 Kode program

4.2.1 Class Pasien_2511532024

```
1. public class Pasien_2511532024 {
2.     String namaPasien_2024;
3.     String penyakit_2024;
4.     int nomorAntrian_2024;
5.     Pasien_2511532024 next_2024;
6.
7.     // konstruktor
8.     public Pasien_2511532024(String namaPasien_2024,
9.         String penyakit_2024, int nomorAntrian_2024) {
10.         this.namaPasien_2024 = namaPasien_2024;
11.         this.penakit_2024 = penyakit_2024;
12.         this.nomorAntrian_2024 = nomorAntrian_2024;
13.         this.next_2024 = null;
14.     }
15.
16.     // getter
17.     public String getNamaPasien_2024() {
18.         return namaPasien_2024;
19.     }
20.     public String getPenyakit_2024() {
21.         return penyakit_2024;
22.     }
23.     public int getNomorAntrian_2024() {
24.         return nomorAntrian_2024;
25.     }
26.
27.     // setter
28.     public void setNext_2024(Pasien_2511532024 next_2024)
29.     {
30.         this.next_2024 = next_2024;
31.     }
32. }
```

4.2.2 Class RumahSakit_2511532024

```
1. import java.util.Scanner;
2.
3. public class Rumahsakit_2511532024 {
4.     static Pasien_2511532024 head_2024 = null;
5.     static int counter_2024 = 0;
6.
7.     // insert (daftar pasien)
8.     public static void insertPasien_2024(String
nama_2024, String penyakit_2024) {
9.         counter_2024++;
10.        Pasien_2511532024 newNode_2024 = new
Pasien_2511532024(nama_2024, penyakit_2024,
counter_2024);
11.
12.        if (head_2024 == null) {
13.            head_2024 = newNode_2024;
14.        } else {
15.            Pasien_2511532024 temp_2024 = head_2024;
16.            while (temp_2024.next_2024 != null) {
17.                temp_2024 = temp_2024.next_2024;
18.            }
19.            temp_2024.next_2024 = newNode_2024;
20.        }
21.
22.        System.out.println("Pasien berhasil
ditambahkan! Nomor Antrian: " + counter_2024);
23.    }
24.
25.    // delete head (panggil pasien)
26.    public static void panggilPasien_2024() {
27.        if (head_2024 == null) {
28.            System.out.println("Antrian kosong!");
29.            return;
30.        }
31.
32.        System.out.println("Memanggil pasien: " +
head_2024.namaPasien_2024 +
33.            " (" + head_2024.penyakit_2024 +
")");
34.        head_2024 = head_2024.next_2024;
35.    }
36.
37.    // display
38.    public static void tampilkanAntrian_2024() {
39.        if (head_2024 == null) {
40.            System.out.println("Antrian kosong!");
41.            return;
42.        }
43.
44.        Pasien_2511532024 temp_2024 = head_2024;
45.        while (temp_2024 != null) {
46.            System.out.println("No: " +
temp_2024.nomorAntrian_2024 +
47.                " | Nama: " +
temp_2024.namaPasien_2024 +
```

```

48.         " | Keluhan: " +
temp_2024.penyakit_2024);
49.         temp_2024 = temp_2024.next_2024;
50.     }
51. }
52.
53. // search (case insensitive)
54. public static void cariPasien_2024(String nama_2024)
{
55.     Pasien_2511532024 temp_2024 = head_2024;
56.     boolean ditemukan = false;
57.
58.     while (temp_2024 != null) {
59.         if
(temp_2024.namaPasien_2024.equalsIgnoreCase(nama_202
4)) {
60.             System.out.println("Pasien
ditemukan: ");
61.             System.out.println("No: " +
temp_2024.nomorAntrian_2024 +
62.                 " | Nama: " +
temp_2024.namaPasien_2024 +
63.                 " | Keluhan: " +
temp_2024.penyakit_2024);
64.             ditemukan = true;
65.             break;
66.         }
67.         temp_2024 = temp_2024.next_2024;
68.     }
69.
70.     if (!ditemukan) {
71.         System.out.println("Pasien tidak
ditemukan.");
72.     }
73. }
74.
75. // Status Antrian
76. public static void statusAntrian_2024() {
77.     if (head_2024 == null) {
78.         System.out.println("Antrian kosong!");
79.         return;
80.     }
81.
82.     int jumlah_2024 = 0;
83.     Pasien_2511532024 temp_2024 = head_2024;
84.
85.     while (temp_2024 != null) {
86.         jumlah_2024++;
87.         temp_2024 = temp_2024.next_2024;
88.     }
89.
90.     System.out.println("jumlah pasien: " +
jumlah_2024);
91.     System.out.println("Pasien terdepan: " +
head_2024.namaPasien_2024);
92. }
93.

```

```

94. // main
95. public static void main(String[] args) {
96.     Scanner sc = new Scanner(System.in);
97.     int pilihan_2024;
98.
99.     do {
100.         System.out.println("\n===
        Antrian Rumah Sakit NIM: 2511532024 ===");
101.         System.out.println("1.
        Daftarkan Pasien (Insert)");
102.         System.out.println("2.
        Panggil Pasien (Delete Head)");
103.         System.out.println("3.
        Tampilkan Antrian (Display)");
104.         System.out.println("4.
        Cari Pasien (Search)");
105.         System.out.println("5. Cek
        Status Antrian");
106.         System.out.println("6.
        Keluar");
107.         System.out.print("\nPilihan : ");
108.         pilihan_2024 =
            sc.nextInt();
109.         sc.nextLine();
110.
111.         switch (pilihan_2024) {
112.             case 1:
113.                 System.out.print("\nMasukkan nama pasien: ");
114.                 String
                    nama_2024 = sc.nextLine();
115.                 System.out.print("Masukkan keluhan: ");
116.                 String
                    keluhan_2024 = sc.nextLine();
117.                 insertPasien_2024(nama_2024, keluhan_2024);
118.                 break;
119.
120.             case 2:
121.                 panggilPasien_2024();
122.                 break;
123.
124.             case 3:
125.                 tampilkanAntrian_2024();
126.                 break;
127.
128.             case 4:
129.                 System.out.print("Masukkan nama yang dicari: ");
130.                 String
                    cari_2024 = sc.nextLine();
131.                 cariPasien_2024(cari_2024);

```

```

132.                                     break;
133.
134.                                     case 5:
135.                                     statusAntrian_2024();
136.                                     break;
137.
138.                                     case 6:
139.
140.                                     System.out.println("Terima kasih.");
141.                                     break;
142.                                     default:
143.
144.                                     System.out.println("Pilihan tidak valid.");
145.                                     }
146.                                     } while (pilihan_2024 != 6);
147.                                 }
148.
149.                                 }

```

4.3 Penjelasan

4.3.1 Class Pasien_2511532024

1. Deklarasi Kelas dan Atribut

```

public class Pasien_2511532024 {
    String namaPasien_2024;
    String penyakit_2024;
    int nomorAntrian_2024;
    Pasien_2511532024 next_2024;
}

```

Bagian ini mendefinisikan kelas Pasien_2511532024 sebagai representasi satu node dalam Single Linked List. Di dalamnya terdapat beberapa atribut, yaitu namaPasien_2024 untuk menyimpan nama pasien, penyakit_2024 untuk menyimpan keluhan atau penyakit, dan nomorAntrian_2024 sebagai identitas urutan pasien dalam antrian. Selain itu, terdapat atribut next_2024 yang berfungsi sebagai pointer untuk menunjuk ke node berikutnya, sehingga setiap objek dapat terhubung membentuk sebuah linked list.

2. Konstruktor

```
public Pasien_2511532024(String namaPasien_2024, String
penyakit_2024, int nomorAntrian_2024) {
    this.namaPasien_2024 = namaPasien_2024;
    this.penyakit_2024 = penyakit_2024;
    this.nomorAntrian_2024 = nomorAntrian_2024;
    this.next_2024 = null;
}
```

Konstruktor digunakan untuk menginisialisasi objek saat pertama kali dibuat. Nilai nama pasien, penyakit, dan nomor antrian diisi sesuai parameter yang diberikan, kemudian pointer next_2024 diatur bernilai null karena pada saat node dibuat, belum terhubung dengan node lain. Ini penting untuk memastikan bahwa node baru tidak langsung “nyasar” ke node lain tanpa kontrol.

3. Method Getter

```
public String getNamaPasien_2024() {
    return namaPasien_2024;
}
public String getPenyakit_2024() {
    return penyakit_2024;
}
public int getNomorAntrian_2024() {
    return nomorAntrian_2024;
}
```

Method getter berfungsi untuk mengambil nilai dari atribut dalam kelas. Dengan adanya getter, data seperti nama pasien, penyakit, dan nomor antrian dapat diakses dari luar kelas tanpa harus langsung menyentuh variabelnya. Ini membantu menjaga prinsip enkapsulasi dalam pemrograman berorientasi objek.

4. Method Setter

```
public void setNext_2024(Pasien_2511532024 next_2024)
{
    this.next_2024 = next_2024;
}
```

Method setter ini digunakan untuk mengatur nilai pointer next_2024, yaitu menghubungkan node saat ini dengan node berikutnya. Method ini sangat penting dalam Single Linked List karena proses penyambungan antar node sepenuhnya bergantung pada pengaturan pointer ini.

4.3.2 Class RumahSakit_2511532024

1. Deklarasi Kelas dan Variabel Global

```
public class Rumahsakit_2511532024 {
    static Pasien_2511532024 head_2024 = null;
    static int counter_2024 = 0;
```

Bagian ini mendefinisikan kelas utama yang berfungsi sebagai pengelola Single Linked List. Variabel head_2024 digunakan sebagai penunjuk ke node pertama dalam antrian, sedangkan counter_2024 berfungsi untuk menyimpan nomor antrian yang akan terus bertambah secara otomatis setiap kali pasien baru ditambahkan.

2. Method Insert Pasien

```
public static void insertPasien_2024(String nama_2024,
String penyakit_2024) {
    counter_2024++;
    Pasien_2511532024 newNode_2024 = new
    Pasien_2511532024(nama_2024, penyakit_2024,
    counter_2024);
    if (head_2024 == null) {
        head_2024 = newNode_2024;
```

```

    } else {
        Pasien_2511532024 temp_2024 = head_2024;
        while (temp_2024.next_2024 != null) {
            temp_2024 = temp_2024.next_2024;
        }
        temp_2024.next_2024 = newNode_2024;
    }
    System.out.println("Pasien berhasil ditambahkan! Nomor
    Antrian: " + counter_2024);
}

```

Method ini digunakan untuk menambahkan pasien ke dalam antrian (insert di belakang/tail). Setiap kali method dipanggil, nilai counter_2024 akan bertambah untuk menghasilkan nomor antrian baru. Node baru dibuat menggunakan constructor kelas Pasien_2511532024. Jika linked list masih kosong, node baru langsung menjadi head. Jika tidak, program akan menelusuri hingga node terakhir, lalu menghubungkan node tersebut dengan node baru melalui pointer next_2024. Ini memastikan konsep FIFO tetap terjaga.

3. Method Panggil Pasien

```

public static void panggilPasien_2024() {
    if (head_2024 == null) {
        System.out.println("Antrian kosong!");
        return;
    }
    System.out.println("Memanggil pasien: " +
    head_2024.namaPasien_2024 +
    " (" + head_2024.penyakit_2024 + ")");
    head_2024 = head_2024.next_2024;
}

```

Method ini berfungsi untuk menghapus node pertama (head), sesuai konsep FIFO. Jika antrian kosong, program menampilkan pesan dan tidak melakukan apa pun. Jika ada data, pasien terdepan ditampilkan, lalu head digeser ke node berikutnya sehingga pasien tersebut keluar dari antrian.

4. Method Tampilan Antrian

```
public static void tampilkanAntrian_2024() {  
    if (head_2024 == null) {  
        System.out.println("Antrian kosong!");  
        return;  
    }  
    Pasien_2511532024 temp_2024 = head_2024;  
    while (temp_2024 != null) {  
        System.out.println("No:          "          +  
temp_2024.nomorAntrian_2024 +  
        " | Nama: " + temp_2024.namaPasien_2024 +  
        " | Keluhan: " + temp_2024.penyakit_2024);  
        temp_2024 = temp_2024.next_2024;  
    }  
}
```

Method ini digunakan untuk menampilkan seluruh isi antrian. Program menelusuri linked list dari head hingga node terakhir menggunakan perulangan, kemudian mencetak data setiap pasien. Jika list kosong, akan ditampilkan pesan bahwa antrian kosong.

5. Method Cari Pasien

```
public static void cariPasien_2024(String nama_2024) {  
    Pasien_2511532024 temp_2024 = head_2024;  
    boolean ditemukan = false;
```

```

while (temp_2024 != null) {
    if
(temp_2024.namaPasien_2024.equalsIgnoreCase(nama_20
24)) {
        System.out.println("Pasien ditemukan: ");
        System.out.println("No: " +
temp_2024.nomorAntrian_2024 +
        " | Nama: " + temp_2024.namaPasien_2024 +
        " | Keluhan: " + temp_2024.penyakit_2024);
        ditemukan = true;
        break;
    }
    temp_2024 = temp_2024.next_2024;
}
if (!ditemukan) {
    System.out.println("Pasien tidak ditemukan.");
}
}

```

Method ini digunakan untuk mencari pasien berdasarkan nama dengan metode case-insensitive. Program menelusuri setiap node dan membandingkan nama menggunakan equalsIgnoreCase(). Jika ditemukan, data pasien ditampilkan. Jika tidak, akan muncul pesan bahwa pasien tidak ditemukan.

6. Method Status Antrian

```

public static void statusAntrian_2024() {
    if (head_2024 == null) {
        System.out.println("Antrian kosong!");
        return;
    }
}

```

```

int jumlah_2024 = 0;
Pasien_2511532024 temp_2024 = head_2024;

while (temp_2024 != null) {
    jumlah_2024++;
    temp_2024 = temp_2024.next_2024;
}

System.out.println("jumlah pasien: " + jumlah_2024);
System.out.println("Pasien terdepan: " +
head_2024.namaPasien_2024);
}

```

Method ini berfungsi untuk menampilkan jumlah total pasien dalam antrian serta informasi pasien terdepan. Program menghitung jumlah node dengan menelusuri seluruh linked list. Jika list kosong, akan ditampilkan pesan bahwa antrian kosong.

7. Method Main

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int pilihan_2024;

    do {
        System.out.println("\n=== Antrian Rumah Sakit NIM:
2511532024 ===");
        System.out.println("1. Daftarkan Pasien (Insert)");
        System.out.println("2. Panggil Pasien (Delete Head)");
        System.out.println("3. Tampilkan Antrian (Display)");
        System.out.println("4. Cari Pasien (Search)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
    }
}

```

```
System.out.print("\nPilihan : ");
pilihan_2024 = sc.nextInt();
sc.nextLine();

switch (pilihan_2024) {
    case 1:
        System.out.print("\nMasukkan nama pasien: ");
        String nama_2024 = sc.nextLine();
        System.out.print("Masukkan keluhan: ");
        String keluhan_2024 = sc.nextLine();
        insertPasien_2024(nama_2024, keluhan_2024);
        break;

    case 2:
        panggilPasien_2024();
        break;

    case 3:
        tampilkanAntrian_2024();
        break;

    case 4:
        System.out.print("Masukkan nama yang dicari: ");
        String cari_2024 = sc.nextLine();
        cariPasien_2024(cari_2024);
        break;

    case 5:
        statusAntrian_2024();
        break;

    case 6:
```

```

        System.out.println("Terima kasih.");
        break;

        default:
            System.out.println("Pilihan tidak valid.");
    }

    } while (pilihan_2024 != 6);
}

```

Bagian ini merupakan pusat eksekusi program yang menggunakan menu interaktif berbasis input pengguna. Program berjalan dalam perulangan do-while hingga pengguna memilih keluar. Setiap pilihan menu akan memanggil method yang sesuai, seperti insert, delete, display, search, dan status antrian. Input diambil menggunakan objek Scanner.

4.4 Output Kode Program

```

=== Antrian Rumah Sakit NIM: 2511532024 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan : 1

Masukkan nama pasien: fulan
Masukkan keluhan: sakit perut
Pasien berhasil ditambahkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511532024 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan : 1

Masukkan nama pasien: fulani
Masukkan keluhan: asam lambung
Pasien berhasil ditambahkan! Nomor Antrian: 2

```



```
=== Antrian Rumah Sakit NIM: 2511532024 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan : 1

Masukkan nama pasien: mamat
Masukkan keluhan: vertigo
Pasien berhasil ditambahkan! Nomor Antrian: 3

=== Antrian Rumah Sakit NIM: 2511532024 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan : 1

Masukkan nama pasien: somat
Masukkan keluhan: radang tenggorokan
Pasien berhasil ditambahkan! Nomor Antrian: 4

=== Antrian Rumah Sakit NIM: 2511532024 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan : 3
No: 1 | Nama: fulan | Keluhan: sakit perut
No: 2 | Nama: fulani | Keluhan: asam lambung
No: 3 | Nama: mamat | Keluhan: vertigo
No: 4 | Nama: somat | Keluhan: radang tenggorokan

=== Antrian Rumah Sakit NIM: 2511532024 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan : 2
Memanggil pasien: fulan (sakit perut)
```

```
=== Antrian Rumah Sakit NIM: 2511532024 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan : 3
No: 2 | Nama: fulani | Keluhan: asam lambung
No: 3 | Nama: mamat | Keluhan: vertigo
No: 4 | Nama: somat | Keluhan: radang tenggorokan

=== Antrian Rumah Sakit NIM: 2511532024 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan : 4
Masukkan nama yang dicari: fulan
Pasien tidak ditemukan.

=== Antrian Rumah Sakit NIM: 2511532024 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan : 4
Masukkan nama yang dicari: mamat
Pasien ditemukan:
No: 3 | Nama: mamat | Keluhan: vertigo

=== Antrian Rumah Sakit NIM: 2511532024 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan : 5
jumlah pasien: 3
Pasien terdepan: fulani

=== Antrian Rumah Sakit NIM: 2511532024 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan : 6
Terima kasih.
```